WanderMind — AI-Powered Smart Travel Planner with Intelligent Memory & Real-Time Web Research

Team:

MUPPALLA ASHISH	AP23110010051
GANTA RAJESH	AP23110010012
KALAKONDA JESWEER	AP23110010054
MALLEMKONDA MANIKANTA	AP23110010081
BONTHA PHANINDRA	AP23110010079

Project Description:

1. Project Overview :

WanderMind is an AI-powered travel planning web application built using Python (Flask), a locally running Ollama LLM (Qwen2.5 / LLaMA 3.2), and the Firecrawl web-scraping service. It provides six intelligent travel features through a unified, memory-aware interface that personalises every interaction based on the user's travel history and preferences.

The platform fetches real-time travel data from WikiVoyage, WikiTravel, and Numbeo using Firecrawl, and combines this with natural language generation from a locally hosted LLM to produce grounded, personalised travel plans, destination profiles, budget breakdowns, packing lists, and destination comparisons.

All AI inference runs locally through Ollama — no user data is sent to external cloud AI services — making WanderMind a privacy-first platform that can operate on consumer hardware.

2. Problem Statement :

Conventional travel planning tools present static itineraries that ignore individual budget constraints, travel style, past trips, and personal interests. Users must visit multiple websites to gather information on costs, attractions, packing, and logistics — a fragmented and time-consuming process. There is no single platform that remembers the user, fetches live data, and generates personalised plans conversationally.

3. Objectives :

The primary objective of this project is to develop an AI-powered travel planning platform that delivers personalised, real-time travel assistance through a locally hosted LLM. The system integrates web scraping for live data retrieval and a persistent memory layer that adapts to each user's travel profile over time.

A key objective is to leverage the Firecrawl API to fetch current information from trusted travel sources and combine it with Ollama LLM inference so that generated plans are grounded in real data rather than outdated training knowledge.

The project also aims to implement the Memento Design Pattern as the memory architecture, demonstrating how a classic software design pattern can solve a practical AI personalisation challenge by persisting user state across sessions.

Additionally, the system demonstrates how to build a multi-feature generative AI application — covering travel planning, research, chat, comparison, budgeting, and packing — using entirely open-source tools running on local hardware without cloud AI subscriptions.

4. Core Technologies Used

Technology	Purpose
Python	Backend logic, service orchestration, and data processing
Flask	Web framework for HTTP routing, template rendering, and session management
Ollama (local LLM server)	Runs quantised open-source LLMs locally for privacy-first AI inference
Qwen2.5 / LLaMA 3.2	LLM models for plan generation, research synthesis, chat,

	and structured JSON extraction
Firecrawl	Real-time web scraping of WikiVoyage, WikiTravel, and Numbeo for live travel data
Memento Pattern (custom)	Persistent user memory — stores trips, preferences, and chat history as JSON
HTML / CSS / JS	Frontend interface — feature pages, dynamic rendering, Clipboard API
python-dotenv	Secure environment variable management via .env file

5. Key Features

Travel Plan Generator

- Users enter origin, destination, budget, duration, interests, and travel style. The backend fetches live travel tips via Firecrawl and calls the Ollama LLM to produce a personalised multi-day itinerary. The completed trip is saved to memory.
- Generates day-by-day itinerary with estimated costs, local tips, and travel logistics
- Incorporates real-time web data from WikiVoyage and Numbeo for accurate cost estimates

Destination Research

- Scrapes up to three sources — WikiVoyage, WikiTravel, and Numbeo — and synthesises a structured JSON destination profile.
- Covers best time to visit, average costs, must-see attractions, culture notes, and safety tips
- Falls back to enriched placeholder content if Firecrawl API key is absent

AI Chat Assistant

- A conversational travel assistant aware of the user's past trips, preferences, and ongoing chat history.
- Retains full conversation context via the memory layer — no need to repeat background information

Destination Comparator

- Compares two destinations side-by-side given budget and interests, with an AI recommendation.
- Returns structured JSON covering cost, climate, activities, best season, and overall recommendation

Budget Optimizer

- Given destination, budget, duration, and priorities, generates an itemised spending breakdown.
- Includes category allocations for accommodation, food, transport, activities, and contingency

Smart Packing List

- Generates a categorised packing checklist customised for destination, season, activities, and duration.
- Categories include documents, clothing, electronics, health, and destination-specific essentials

Memory System (My Trips)

- Implemented via the Memento Design Pattern — persists user profile, past trips, and chat history in memory_store.json across all sessions.

6. Target Users

- Travellers who want personalised, AI-generated itineraries without paying for cloud AI subscriptions
- Budget-conscious explorers who need accurate cost estimates and optimised spending plans
- Privacy-focused users who prefer local AI inference over sending data to external services
- Students and developers learning how to integrate LLMs, web scraping, and design patterns in a real application

7. Real-World Applications

WanderMind bridges the gap between static travel information and intelligent, personalised planning. By combining live web data retrieval with local LLM inference and persistent user memory, the system delivers a genuinely context-aware travel assistant. It is relevant for independent travellers, travel agencies building internal tools, and developers exploring production-grade generative AI integration with open-source models.

APPLICATION SCENARIOS / USE CASES :

Scenario 1: Planning a Family Holiday

A family of four wants to plan a 7-day trip from Hyderabad to Goa with a budget of ₹80,000. They use the Travel Plan Generator, entering their origin, destination, budget, duration, and interests (beaches, local cuisine). Firecrawl fetches current data from WikiVoyage and Numbeo, and the LLM generates a day-by-day itinerary with hotel suggestions, daily costs, and activity recommendations. The trip is saved to memory so future comparisons reference it.

Scenario 2: Choosing Between Two Destinations

A solo traveller is undecided between Manali and Shimla for a 5-day winter trip. They use the Destination Comparator, entering both destinations with their budget and interests. The system returns a structured JSON comparison covering climate, costs, activities, and best season, and recommends Manali for adventure seekers or Shimla for a relaxed cultural experience.

Scenario 3: Budget-Constrained Trip Planning

A student has USD 1,000 for 7 days in Thailand. They use the Budget Optimizer to get an itemised breakdown — accommodation, food, transport, activities — with money-saving tips such as using local markets and overnight buses. The system also generates a packing list tailored to tropical weather and budget travel style.

Scenario 4: Ongoing Travel Research

A travel blogger regularly uses the AI Chat Assistant to research destinations across multiple sessions. Because WanderMind remembers previous conversations and the user's stated home city, interests, and travel style, each new chat session builds on prior context. The blogger asks follow-up questions without re-explaining their preferences each time.

TECHNICAL ARCHITECTURE :

- **Frontend Layer**

The frontend consists of nine Jinja2 HTML templates (index, planner, result, research, chat, compare, budget, packing, trips) served by Flask. Each feature page collects user inputs through HTML forms and sends AJAX POST requests to the corresponding /api/* endpoint. JavaScript renders the JSON responses dynamically without full page reloads. Styling is handled by a shared style.css (~1,600 lines) and script.js provides shared utilities including copy-to-clipboard functionality.

- **Backend Layer**

The backend is a Flask application (app.py) that defines all routing logic. It instantiates three service singletons at startup: OllamaService (wraps the local Ollama REST API), FirecrawlService (wraps the Firecrawl SDK), and the MemoryCaretaker (loads the JSON memory store). A decorator _ollama_required guards all AI endpoints and returns HTTP 503 if Ollama is unavailable, ensuring graceful error handling.

- **AI Layer (OllamaService)**

OllamaService communicates with the Ollama server running at http://localhost:11434. It exposes two low-level primitives: _call_json() for structured JSON responses (strips markdown fences and extracts the first valid JSON object via regex) and _call_text() for plain-text responses. Six high-level methods correspond to the six application features, each with a carefully engineered system prompt that defines the expected output schema.

● **Web Data Layer (FirecrawlService)**

FirecrawlService scrapes WikiVoyage, WikiTravel, and Numbeo on demand using the Firecrawl SDK. The scraped text is truncated to a configured character limit and passed into the Ollama prompt to ground the LLM response in real data. When the API key is absent, the service returns structured placeholder content so the application remains fully functional.

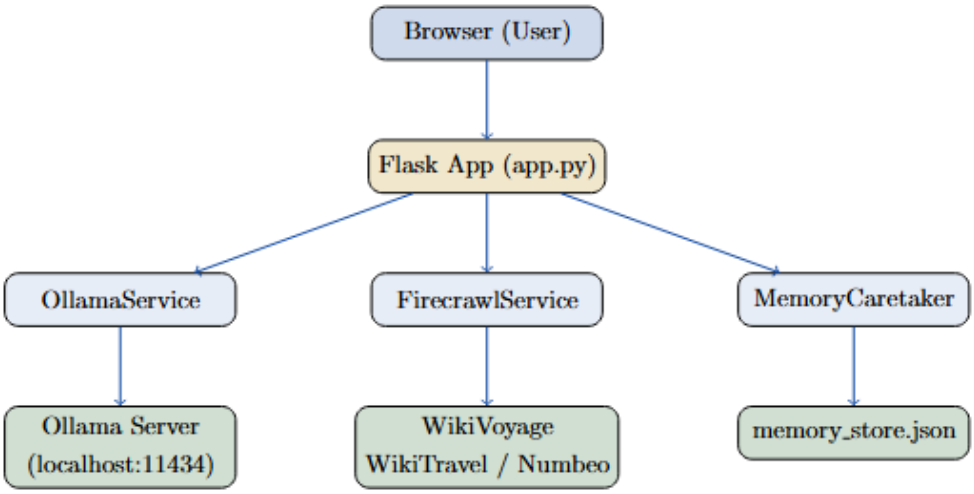
● **Memory Layer (Memento Pattern)**

The memory system implements the Memento Design Pattern with three classes: TravelMemento (immutable state snapshot), UserProfile (Originator — owns mutable user state), and MemoryCaretaker (Caretaker — manages snapshots and persists them to memory_store.json). Every AI call receives a context_summary() string injected into the system prompt, enabling personalisation without the user repeating their preferences.

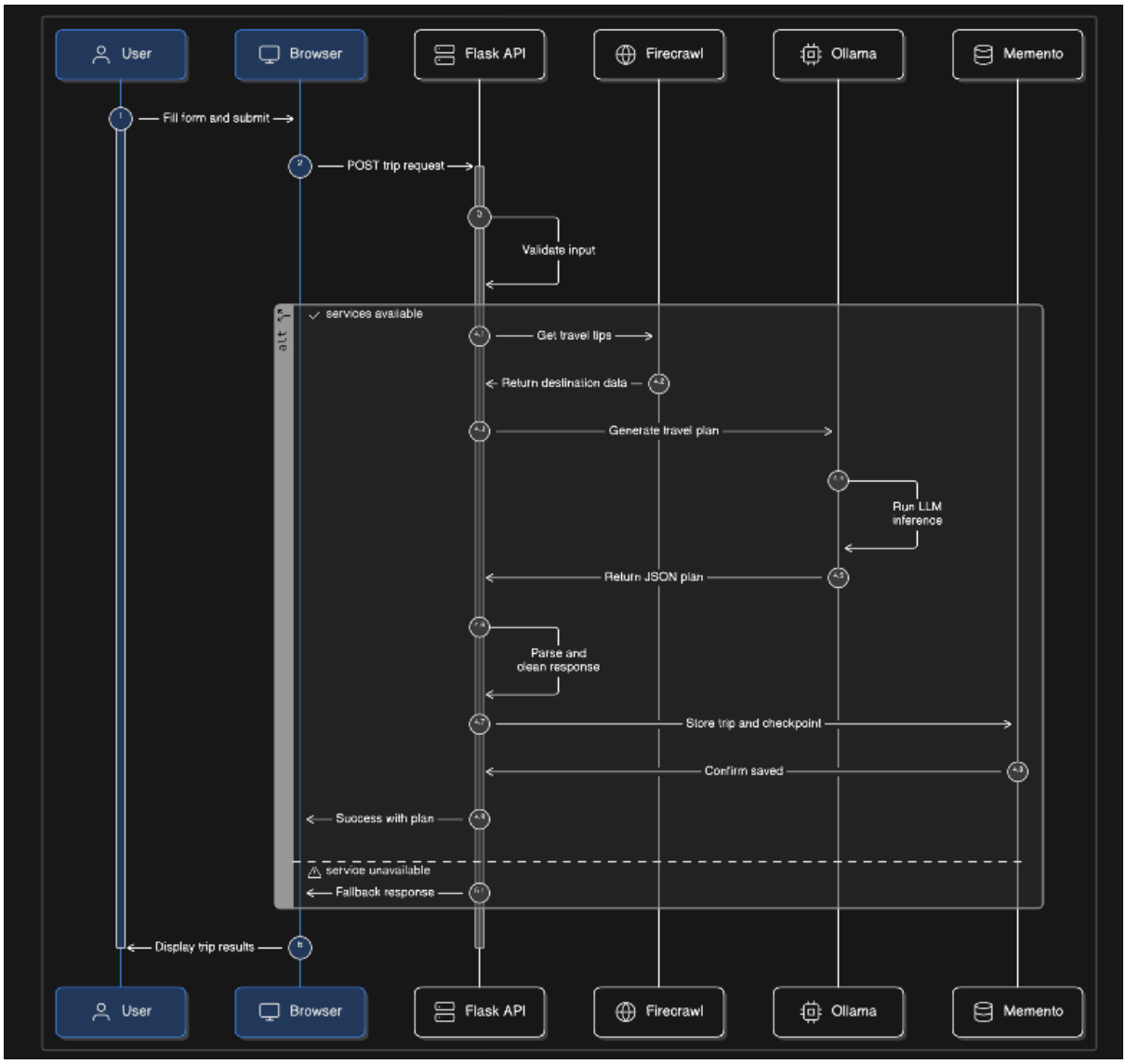
● **Deployment Environment**

The application runs locally with a single command: python app.py. Ollama operates as a separate background server. For production deployment, the Flask application can be served via Gunicorn, and Ollama can be run on a cloud VM with optional GPU support for faster inference. The architecture supports model switching via environment variable configuration.

High-Level Architecture Diagram



Component Interaction Diagram



Pre-requisites:

Software Requirements

- Python: Version 3.10 or higher. Ensure 'Add Python to PATH' is enabled during installation on Windows.
- Ollama: Download and install from https://ollama.com/download. Required to run the LLM locally. After installation run: ollama pull qwen2.5:0.5b
- pip: Python package manager bundled with Python 3.10+. Used to install all project dependencies.
- IDE: Visual Studio Code with the Python extension or Cursor IDE for AI-assisted development.
- Browser: Modern web browsers such as Chrome, Edge, or Firefox supporting ES6, async/await, and Clipboard API.
- Git: Version control system for project management and source code tracking (optional but recommended).

Libraries / Frameworks / Dependencies

Install required dependencies using:

```
pip install flask python-dotenv requests firecrawl-py
```

- Flask: Web framework used for REST API routing, handling HTTP requests, and rendering Jinja2 templates.
- requests: HTTP client library used by OllamaService to communicate with the local Ollama REST API at localhost:11434.

- firecrawl-py: Official Firecrawl Python SDK for web scraping. Gracefully falls back if the API key is absent.
- python-dotenv: Loads environment variables from the .env file to keep API keys out of source code.

Hardware Requirements

- CPU: Minimum Intel Core i5 or AMD Ryzen 5 processor with at least 2 GHz clock speed. Qwen2.5:0.5B runs comfortably on modern CPUs without a GPU.
- RAM: Minimum 8 GB recommended. Qwen2.5:0.5B requires ~1 GB; LLaMA 3.2:3B requires ~4 GB. More RAM improves inference speed.
- Storage: At least 3 GB free disk space for the Ollama model file, Python packages, and project files.
- GPU (Optional): NVIDIA GPU with CUDA support. Ollama automatically uses GPU acceleration when available, significantly improving response speed.
- Network: Internet connection required for the initial Ollama model pull, Firecrawl API calls, and loading Google Fonts in the UI.

PRIOR KNOWLEDGE REQUIRED

Programming Language Knowledge

- Strong understanding of Python fundamentals (variables, loops, functions, modules)
- Knowledge of object-oriented programming (OOP) concepts — classes, instances, methods
- Familiarity with handling JSON data, working with dictionaries, and calling REST APIs from Python
- Basic understanding of error handling and debugging techniques (try/except, print debugging)

Framework Basics

- Understanding of Flask framework for building web applications and RESTful APIs
- Knowledge of Flask routing, request/response handling, and jsonify()
- Familiarity with Jinja2 template rendering using render_template()
- Basic understanding of Flask session management and application configuration

AI / LLM Knowledge

- Basic understanding of large language models (LLMs) and how they generate text
- Familiarity with the Ollama platform and how to pull and run open-source models locally
- Understanding of prompt engineering — system prompts, user prompts, and structured JSON extraction
- Awareness of prompt design for constraining LLM output to specific JSON schemas

Design Patterns

- Understanding of the Memento Design Pattern (Originator, Memento, Caretaker) as used in the memory layer
- Basic awareness of software design principles — separation of concerns and modularity

Frontend Basics

- HTML/CSS for page structure and styling
- JavaScript ES6 — fetch API, async/await, DOM manipulation for dynamic rendering
- Basic understanding of AJAX for sending and receiving JSON data without page reloads

7. PROJECT OBJECTIVES

Technical Objectives

- Design and develop a Flask-based multi-feature AI travel planning application with six distinct modules
- Integrate a locally running Ollama LLM (Qwen2.5 / LLaMA 3.2) for privacy-preserving AI inference
- Implement real-time web data retrieval using the Firecrawl API to enrich AI responses with live travel information
- Build a persistent user memory system using the Memento Design Pattern stored as JSON
- Develop modular service classes (OllamaService, FirecrawlService) with clean separation from routing logic

Performance Objectives

- Achieve responsive AI outputs for all six features with acceptable inference latency on local consumer hardware
- Ensure graceful degradation — both Firecrawl and Ollama have fallback modes when unavailable
- Maintain consistent JSON parsing and structured error responses across all API endpoints
- Ensure memory operations (read/write) are fast and do not impact API response times

Deployment Objectives

- Enable easy local deployment with a single command: python app.py
- Support configuration via .env file for API keys and model selection without code changes

- Ensure cross-browser compatibility for the web frontend (Chrome, Edge, Firefox)
- Maintain clear setup documentation enabling a new developer to run the project within minutes

Learning Outcomes

- Gain hands-on experience integrating a local LLM (Ollama) into a production-style web application
- Understand how to combine web scraping (Firecrawl) with LLM prompting for Retrieval-Augmented Generation (RAG)
- Learn to apply the Memento Design Pattern to solve a real AI personalisation problem
- Develop skills in Flask API design, prompt engineering, and JSON-structured LLM output parsing
- Experience building a coherent multi-feature AI product with a clean user interface

Project WorkFlow:

Milestone 1: Requirement Analysis & System Design

- Activity 1.1 — Problem Definition: Identified the core problem of fragmented, non-personalised, cloud-dependent travel planning tools.
- Activity 1.2 — Feature Specification: Defined the six core features and their input/output schemas.
- Activity 1.3 — Technology Selection: Evaluated Flask vs FastAPI, Ollama vs cloud APIs, and selected the Memento pattern for memory management.
- Activity 1.4 — Architecture Design: Designed the layered service architecture with singleton pattern.
- Activity 1.5 — API Schema Design: Defined all six JSON output schemas for LLM prompt engineering.

Milestone 2: Environment Setup & Initial Configuration

- Activity 2.1 — Project Structure: Created the travelplanner/ directory with services/, memory/, templates/, static/ subdirectories.
- Activity 2.2 — Flask Application: Set up app.py with page routes and service singletons.
- Activity 2.3 — Environment Configuration: Implemented .env loading via python-dotenv.
- Activity 2.4 — Ollama Integration: Built the OllamaService class with connectivity check, chat, call json, and call text methods.

- Activity 2.5 — Route Guards: Implemented the ollama required decorator for automatic 503 responses when Ollama is unavailable

```
PS C:\Users\Dell\GenAI\travelplanner> python app.py
[Ollama] ✅ Connected — model: qwen2.5:0.5b (local)
[Firecrawl] ✅ Connected

🌍 WanderMind starting on http://localhost:5000

 * Serving Flask app 'app'
 * Debug mode: on
```

Milestone 3: Ollama & Firecrawl Integration

- Activity 3.1: Install Ollama and pull the model (qwen2.5:0.5b)
- Activity 3.2: Implement OllamaService with JSON and text call modes
- Activity 3.3: Implement FirecrawlService with WikiVoyage / WikiTravel / Numbeo scraping and fallback

Milestone 4: Core Feature Development

- Activity 4.1: Travel Plan Generator — Firecrawl + Ollama pipeline, memory save
- Activity 4.2: Destination Research — multi-source scraping + LLM synthesis
- Activity 4.3: AI Chat Assistant — conversation history + memory context injection
- Activity 4.4: Destination Comparator, Budget Optimizer, Packing List — structured JSON prompts

Milestone 5: Memory System Development

- Activity 5.1: Implement TravelMemento (immutable snapshot)
- Activity 5.2: Implement UserProfile (Originator) with state mutations
- Activity 5.3: Implement MemoryCaretaker with JSON persistence and context_summary()
- Activity 5.4: Integrate memory into all six feature API endpoints

Milestone 6: Frontend Development

- Activity 6.1: Build nine Jinja2 HTML templates for all feature pages.
- Activity 6.2: Develop shared style.css and script.js for consistent UI and AJAX calls.

Milestone 7: Testing & Validation

- Activity 7.1: Functional Test Case Execution
- Activity 7.2: API Endpoint Testing (all six /api/* routes)
- Activity 7.3: Ollama Fallback Testing (503 handling when Ollama is stopped)
- Activity 7.4: Memory Persistence Testing (verify memory_store.json across restarts)

Milestone 8: Deployment

- Activity 8.1: Deployment Architecture Setup (local and optional cloud)
- Activity 8.2: Hosting Configuration — Flask dev server or Gunicorn for production
- Activity 8.3: Deployment Implementation & Monitoring
- Activity 8.4: Production Deployment Considerations (environment variables, model selection)

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

Identify the need for a privacy-first, memory-aware travel planning platform that fetches live web data and generates personalised itineraries through a locally hosted LLM. Address the limitations of static travel tools that ignore user history and force manual information gathering across multiple websites. Design a solution integrating Flask routing, Ollama LLM inference, Firecrawl web scraping, and a Memento-based memory layer to deliver a coherent, multi-feature AI travel assistant.

Activity 1.2: Functional Requirements

FR-01: Allow users to generate personalised multi-day travel plans based on origin, destination, budget, duration, interests, and travel style.

FR-02: Provide destination research by scraping WikiVoyage, WikiTravel, and Numbeo and synthesising results via the LLM.

FR-03: Enable a conversational AI chat assistant that retains memory across messages within and across sessions.

FR-04: Support side-by-side destination comparison with structured JSON output and AI recommendation.

FR-05: Offer an AI-driven budget optimiser that produces itemised spending allocations across categories.

FR-06: Generate categorised packing lists tailored to destination, season, activities, and travel style.

FR-07: Persist user profile, planned trips, visited destinations, and chat history across sessions via JSON memory store.

FR-08: Expose RESTful API endpoints for all features to support frontend AJAX calls with structured JSON responses.

Activity 1.3: Non-Functional Requirements

NFR-01 Performance: API responses must be generated within acceptable latency on local consumer hardware without GPU.

NFR-02 Reliability: All endpoints must return structured JSON with clear error messages on failure; no unhandled exceptions.

NFR-03 Privacy: AI inference runs locally — no user travel data is transmitted to external cloud AI services.

NFR-04 Security: API keys (Firecrawl) are stored in .env and never hard-coded or committed to source control.

NFR-05 Maintainability: Service classes are modular, independently testable, and separated from routing logic.

NFR-06 Usability: All six features are accessible via a clear web interface; errors are displayed with friendly messages.

Activity 1.4: System Design Decisions & Technology Stack Selection

Backend Framework: Use Flask for lightweight web routing and Jinja2 template rendering, avoiding heavyweight frameworks.

AI Engine: Use Ollama for local LLM execution — supports Qwen2.5, LLaMA, Mistral, DeepSeek, and others via model name configuration.

Web Data: Use Firecrawl for managed web scraping with automatic content extraction from travel sites.

Memory: Use the Memento Design Pattern with JSON file persistence — no database dependency, easy to inspect and debug.

Frontend: Use vanilla HTML/CSS/JavaScript with Jinja2 templates — avoids heavy framework overhead for a focused educational project.

Configuration: Use python-dotenv for secure .env-based configuration of API keys and model selection.

Milestone 2: Environment Setup & Initial Configuration

Activity 2.1: Development Environment Setup

- Install Python 3.10 from the official Python website. On Windows, ensure 'Add Python to PATH' is enabled during installation.
- Install Visual Studio Code with the Python extension, or Cursor IDE for AI-assisted development and debugging.
- Create the main project directory: mkdir travelplanner and cd travelplanner
- Create and activate a virtual environment: python -m venv venv
- Activate: venv\Scripts\activate (Windows) or source venv/bin/activate (macOS/Linux)
- Install Ollama from https://ollama.com/download, then pull the model: ollama pull qwen2.5:0.5b

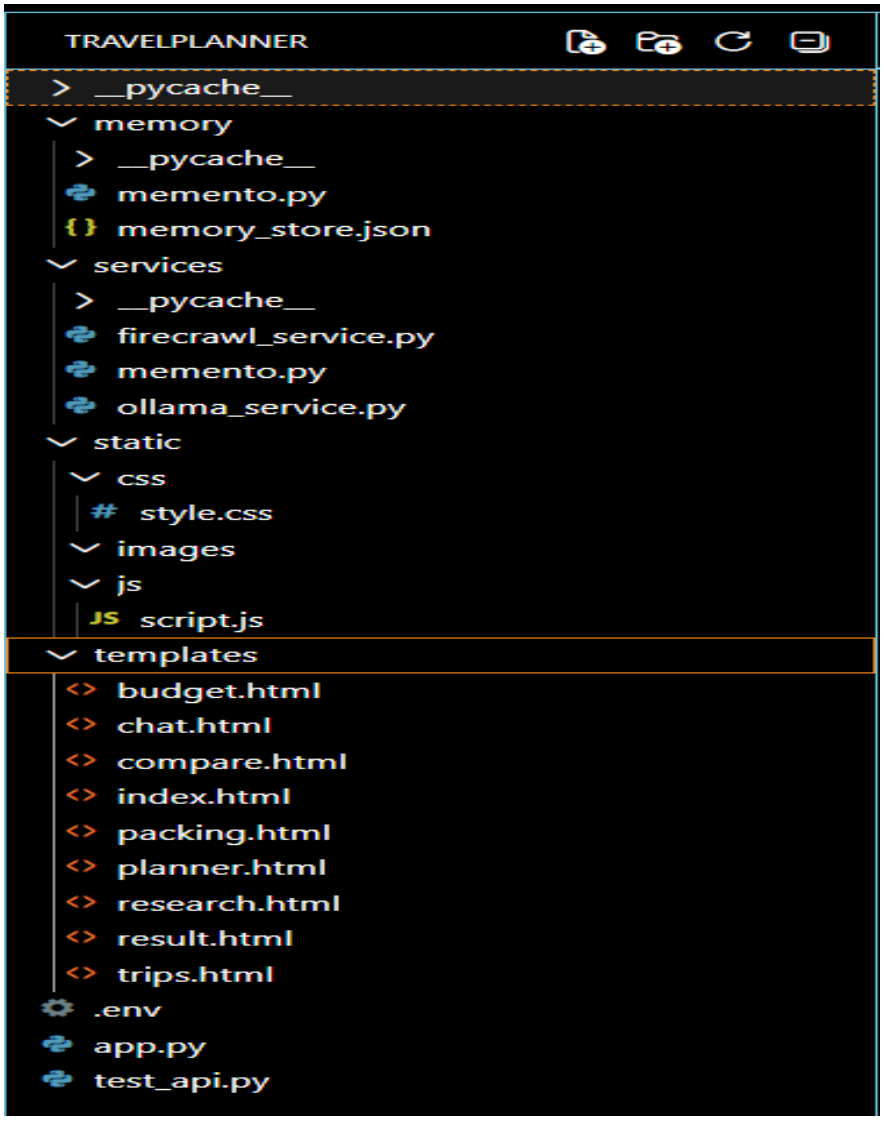
Activity 2.2: Dependency Installation

Install all required Python libraries to support web framework, LLM integration, web scraping, and configuration management.

```
pip install flask python-dotenv requests firecrawl-py
```

Activity 2.3: Project Structure Creation

Organise the project into the following clean, modular folder structure:



Activity 2.4: Configuration Setup

Create a .env file in the project root. This file must never be committed to source control.

```
.env
FIRECRAWL_API_KEY=fc-75e8d64e36354dab9576a62f249d44c9
OLLAMA_MODEL=qwen2.5:0.5b
```

Optionally set OLLAMA_HOST if the Ollama server runs on a non-default address (default: http://127.0.0.1:11434). The Firecrawl API key is available from https://firecrawl.dev. The application runs in fallback mode if the key is absent.

Milestone 3: Ollama & Firecrawl Integration

Activity 3.1: Install Ollama and Pull the Model

Step 1: Download Ollama from https://ollama.com/download and complete installation.

Step 2: Verify the Ollama server starts automatically. If not, run: ollama serve

Step 3: Pull the configured model: ollama pull qwen2.5:0.5b

Step 4: Verify the model is available: ollama list — the model should appear in the output.

Step 5: The OllamaService class checks model availability on startup and prints a status message.

Alternative models supported: llama3.2, mistral, qwen2.5:1.5b, deepseek-r1. Set OLLAMA_MODEL in .env to switch.

Activity 3.2: Implement OllamaService

OllamaService (services/ollama_service.py) wraps the Ollama /api/chat endpoint. It exposes two core primitives:

- _call_json(system, user): Sends a system+user prompt, strips markdown fences from the response, extracts the first valid JSON object via regex, and parses it. Used by all structured-output features.
- _call_text(system, user): Sends a system+user prompt and returns plain text. Used by the Chat Assistant.

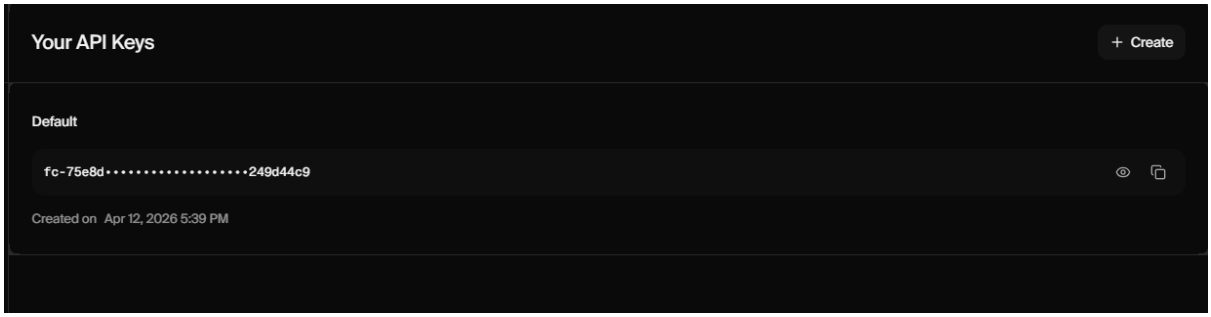
Six high-level feature methods build domain-specific prompts injecting web data and memory context before calling _call_json() or _call_text(): generate_travel_plan(), research_destination(), chat(), compare_destinations(), optimize_budget(), generate_packing_list().

Activity 3.3: Implement FirecrawlService

FirecrawlService (services/firecrawl_service.py) initialises FirecrawlApp with the API key from .env. It provides two public methods:

- get_travel_tips(destination): Scrapes WikiVoyage for general travel guidance on the destination.
- research_destination(destination): Scrapes WikiVoyage, WikiTravel, and Numbeo and returns a dict with source name → scraped text. The OllamaService concatenates these texts and injects them into the research prompt.

When FIRECRAWL_API_KEY is absent, both methods return structured placeholder text so all six features continue to function without real web data.



Milestone 4: Core Feature Development

Activity 4.1: Travel Plan Generator

The /api/plan endpoint validates five required fields (origin, destination, budget, duration, interests), calls firecrawl.get_travel_tips() for live data, passes the result plus memory context into ollama.generate_travel_plan(), saves the trip to memory, and returns the plan JSON. The /result template renders the structured plan as a formatted page with copy-to-clipboard functionality.

Activity 4.2: Destination Research

The /api/research endpoint calls firecrawl.research_destination() to fetch multi-source data, concatenates all scraped texts, and passes them to ollama.research_destination() which synthesises a structured JSON profile. The response includes data_sources (count of scraped sources) and firecrawl_status (enabled/disabled) so the frontend can display data provenance.

Activity 4.3: AI Chat Assistant

The /api/chat endpoint retrieves the full conversation history from memory (memory.get_chat_history()), passes it along with the memory context summary to ollama.chat(), stores the new user message and assistant reply in memory, and returns the reply. The chat template displays the conversation thread and sends new messages via AJAX.

Activity 4.4: Comparator, Budget Optimiser, and Packing List

The /api/compare endpoint validates dest_a and dest_b and calls ollama.compare_destinations() with budget, duration, and interests, returning a structured JSON comparison object. The /api/budget endpoint validates destination and total_budget, then calls ollama.optimize_budget() returning an itemised budget breakdown. The /api/packing endpoint validates destination and calls ollama.generate_packing_list() returning a categorised packing checklist.

Milestone 5: Memory System Development

Activity 5.1: TravelMemento (Immutable Snapshot)

TravelMemento stores a deep copy of the UserProfile state along with a timestamp and optional label. It is immutable — no methods modify the internal state after creation. The to_dict() and from_dict() methods enable JSON serialisation and deserialisation for file persistence.

Activity 5.2: UserProfile (Originator)

UserProfile owns the mutable user state: name, home_city, budget_preference, interests, travel_style, dietary_preferences, visited_destinations, planned_trips, and chat_context. The update() method accepts keyword arguments matching state keys. The add_trip() method appends a trip dict (with timestamp) to planned_trips and adds the destination to visited_destinations, keeping the last 20 trips.

Activity 5.3: MemoryCaretaker

MemoryCaretaker manages the TravelMemento history and persists it to memory_store.json using JSON serialisation. On initialisation it loads any existing memory file. The save() method creates a new TravelMemento from the current UserProfile state and appends it to history. The context_summary() method returns a concise string representation of the user's profile and recent trips that is injected into every LLM prompt.

Activity 5.4: Memory Integration into API Endpoints

All six feature API endpoints call memory.context_summary() before constructing the LLM prompt. The Travel Plan Generator and Chat endpoints additionally call memory.add_trip() and memory.add_chat() after successful LLM responses to update persistent state. The /api/memory GET endpoint exposes the full memory state for the My Trips dashboard page.

Milestone 6: Frontend Development

Activity 6.1: Jinja2 Template Development

Nine HTML templates are built using Jinja2 with Flask: index.html (home with feature cards), planner.html (travel plan form), result.html (rendered itinerary), research.html (destination profile), chat.html (conversation interface), compare.html (comparison form and result), budget.html (budget input and breakdown), packing.html (packing list output), trips.html (memory viewer with profile and trip history).

Activity 6.2: Shared CSS and JavaScript

style.css (~1,600 lines) defines a consistent visual theme across all nine pages: colour palette, card layouts, form styles, loading indicators, and responsive design. script.js provides shared

utilities: AJAX form submission handlers, JSON response rendering, copy-to-clipboard button functionality, and loading state management.

Milestone 7: Testing & Validation

Activity 7.1: Functional Test Case Execution

Test Case	Input	Expected Output
Travel Plan Generation	Origin: Hyderabad, Dest: Goa, 5 days, ₹30,000	Structured JSON itinerary with daily plan, costs, and tips
Destination Research	Destination: Paris	JSON profile: attractions, costs, best time, culture notes
AI Chat	'Best budget hotels in Bangkok?'	Contextual reply referencing user memory if available
Destination Compare	Manali vs Shimla, 5 days	Side-by-side JSON comparison with AI recommendation
Budget Optimiser	Thailand, USD 1500, 7 days	Itemised breakdown with saving tips
Packing List	Iceland, 10 days, Winter	Categorised checklist with destination-specific items
Memory Persistence	Generate plan → restart server → open /trips	Previous trip and profile visible from memory_store.json
Ollama Fallback	Stop Ollama server, submit any AI form	HTTP 503 with 'Ollama service not available' message
Firecrawl Fallback	Remove API key from .env,	Returns placeholder travel data

	run research	without crashing

Activity 7.2: API Endpoint Testing

All nine API endpoints were tested using the test_api.py script and browser developer tools. Each endpoint was verified for correct JSON response structure on valid input, appropriate HTTP 400 errors on missing required fields, HTTP 503 when Ollama is stopped, and correct fallback behaviour when Firecrawl is disabled.

Activity 7.3: Memory Integrity Testing

The memory_store.json file was inspected before and after plan generation, chat sessions, and server restarts to confirm that new trips, profile updates, and conversation history are correctly serialised and reloaded. Memory cleanup via /api/memory/clear was also verified.

Activity 7.4: Performance Evaluation

Response time measurements on Intel Core i5, 16 GB RAM, Qwen2.5:0.5B (no GPU):

Feature	Avg. Response Time	Notes
Travel Plan Generator	15–45 seconds	Includes Firecrawl scraping (~3–8s overhead)
Destination Research	10–30 seconds	Multi-source scraping adds latency
AI Chat	5–15 seconds	Shorter prompts yield faster responses
Destination Compare	10–25 seconds	JSON schema enforcement adds reliability

Budget Optimiser	8–20 seconds	Consistent across destinations
Packing List	5–15 seconds	Fastest feature — simplest prompt structure

Milestone 8: Deployment

Activity 8.1: Deployment Architecture Setup

For local deployment, the Flask development server is started with python app.py and the application is accessible at http://localhost:5000. Ollama must be running separately (ollama serve). For production deployment, Gunicorn replaces the Flask dev server: gunicorn -w 4 app:app

Activity 8.2: Hosting Configuration

The application can be deployed on any cloud VM with sufficient RAM for the Ollama model. Recommended cloud options with GPU support for faster inference: Runpod (NVIDIA A40), Lambda Labs (RTX 3090), or any provider supporting CUDA. The OLLAMA_HOST environment variable allows the Ollama server to run on a separate machine from the Flask application.

Activity 8.3: Deployment Implementation & Monitoring

Ensure Ollama is running and the configured model is pulled before starting Flask. Set all environment variables in .env or as system environment variables. Use a process manager such as systemd or supervisord to keep both the Flask app and Ollama server running persistently. Monitor Ollama inference logs and Flask request logs for errors.

Activity 8.4: Production Deployment Considerations

- Replace app.secret_key = os.urandom(24) with a fixed strong secret key in production to preserve session state across restarts.
- Move memory_store.json to a persistent volume if deploying in a containerised environment.
- Add CORS headers if the Flask API is consumed by a separate frontend deployment.
- Consider rate limiting on the /api/* endpoints to prevent excessive LLM invocations.

PROJECT STRUCTURE

Clean Folder Structure

```
travelplanner/
│
├── app.py
├── .env
├── test_api.py
│
├── memory/
│   ├── memento.py
│   └── memory_store.json
│
├── services/
│   ├── ollama_service.py
│   └── firecrawl_service.py
│
├── static/
│   ├── css/style.css
│   └── js/script.js
│
└── templates/
    ├── index.html
```

```
├── planner.html
├── result.html
├── research.html
├── chat.html
├── compare.html
├── budget.html
├── packing.html
└── trips.html
```

Folder Explanations

app.py

- Flask application entry point. Defines nine page routes and nine API routes.
- Instantiates OllamaService, FirecrawlService, and MemoryCaretaker as singletons at startup.
- Implements the _ollama_required decorator that returns HTTP 503 when Ollama is unavailable.
- Each API endpoint validates input, orchestrates service calls, and returns structured JSON.
- Memory is updated after successful plan generation and chat interactions.

memory/memento.py

- Implements the complete Memento Design Pattern with three classes.

- TravelMemento: immutable snapshot with deep-copy state, timestamp, and JSON serialisation.
- UserProfile: Originator with mutable state — profile fields, trips, visited destinations, chat context.
- MemoryCaretaker: manages snapshot history, persists to memory_store.json, provides context_summary().
- get_memory() factory function returns a singleton MemoryCaretaker instance used across the app.

services/ollama_service.py

- Wraps the Ollama /api/chat REST endpoint using the requests library.
- _check_connection() verifies the model is available locally on startup.
- _call_json() handles JSON extraction with markdown fence stripping and regex fallback.
- Six high-level methods build domain-specific prompts injecting web data and memory context.
- Supports any Ollama-compatible model via the OLLAMA_MODEL environment variable.

services/firecrawl_service.py

- Wraps the FirecrawlApp SDK for managed web scraping.
- Scrapes WikiVoyage, WikiTravel, and Numbeo for destination-specific travel data.
- Returns structured placeholder content when FIRECRAWL_API_KEY is absent.
- Truncates scraped text to configured character limits to avoid exceeding LLM context windows.

- Provides get_travel_tips() for plan generation and research_destination() for multi-source research.

templates/

- Nine Jinja2 HTML templates — one per application page.
- All templates extend a shared layout with consistent navigation and styling.
- Feature pages use HTML forms with JavaScript AJAX submission to avoid full page reloads.
- Result rendering is handled dynamically in JavaScript by parsing the API JSON response.
- trips.html renders the memory state (profile, visited destinations, planned trips, chat history).

static/

- css/style.css: Application-wide stylesheet (~1,600 lines) defining layout, colours, cards, and responsive design.
- js/script.js: Shared JavaScript utilities — AJAX handlers, response renderers, copy-to-clipboard.

.env

- Stores sensitive configuration data outside source code.
- FIRECRAWL_API_KEY: Required for live web scraping. App runs in fallback mode without it.
- OLLAMA_MODEL: Selects the Ollama model (default: qwen2.5:0.5b). Change to llama3.2 for a more capable model.

- OLLAMA_HOST: Optional override for the Ollama server URL (default: http://127.0.0.1:11434).
- Must not be committed to source control — add .env to .gitignore.

test_api.py

- Minimal test script for verifying API endpoint availability and response structure.
- Tests basic connectivity to the Flask server and validates JSON response formats.
- Run after starting the application to confirm all services are wired correctly.

RESULTS

● System Output

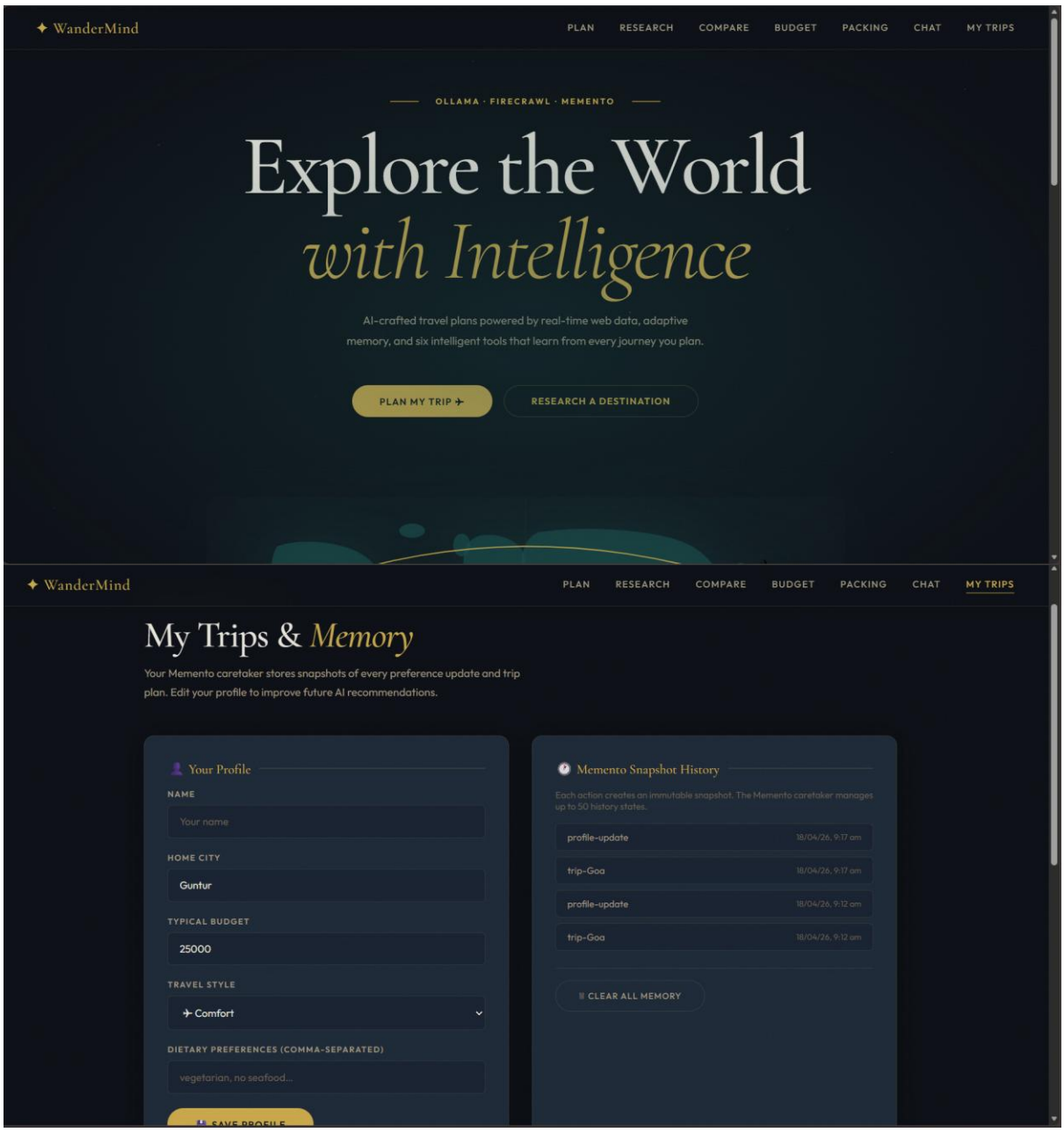
WanderMind successfully generates personalised travel content across all six feature modules. The Travel Plan Generator produces structured multi-day itineraries grounded in real web data. The Destination Research feature returns comprehensive JSON profiles synthesised from multiple scraped sources. The Budget Optimiser delivers itemised spending plans tailored to the user's stated priorities and travel style. Memory-based personalisation improves output quality with each subsequent interaction.

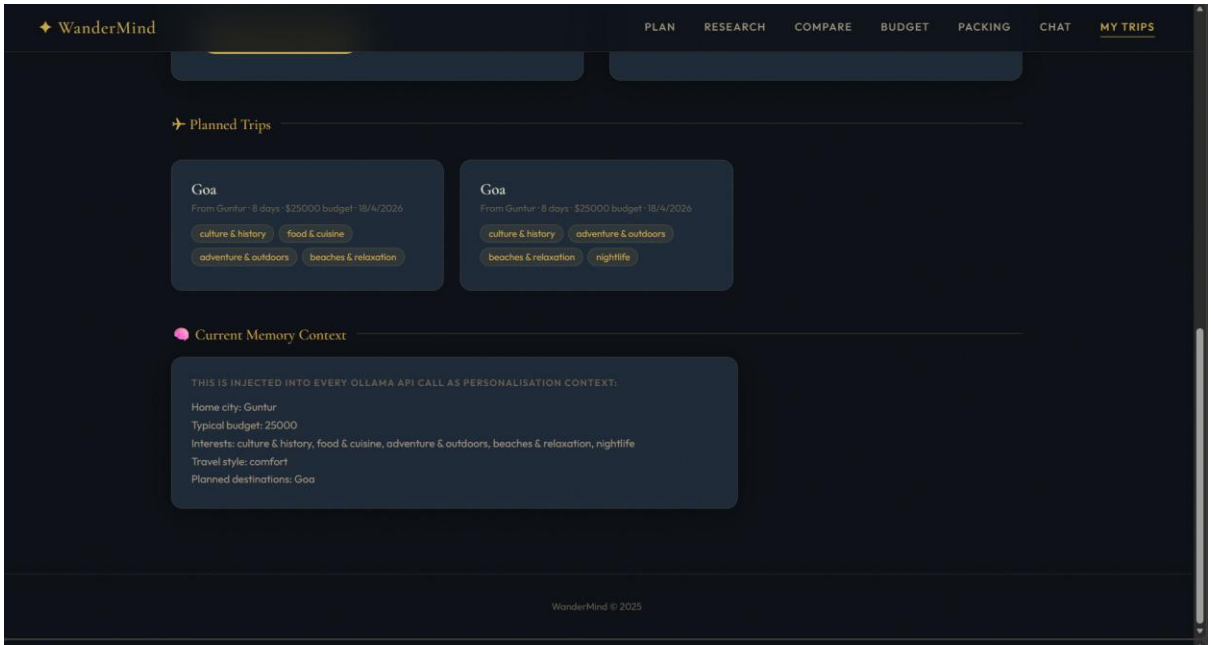
● Performance Evaluation

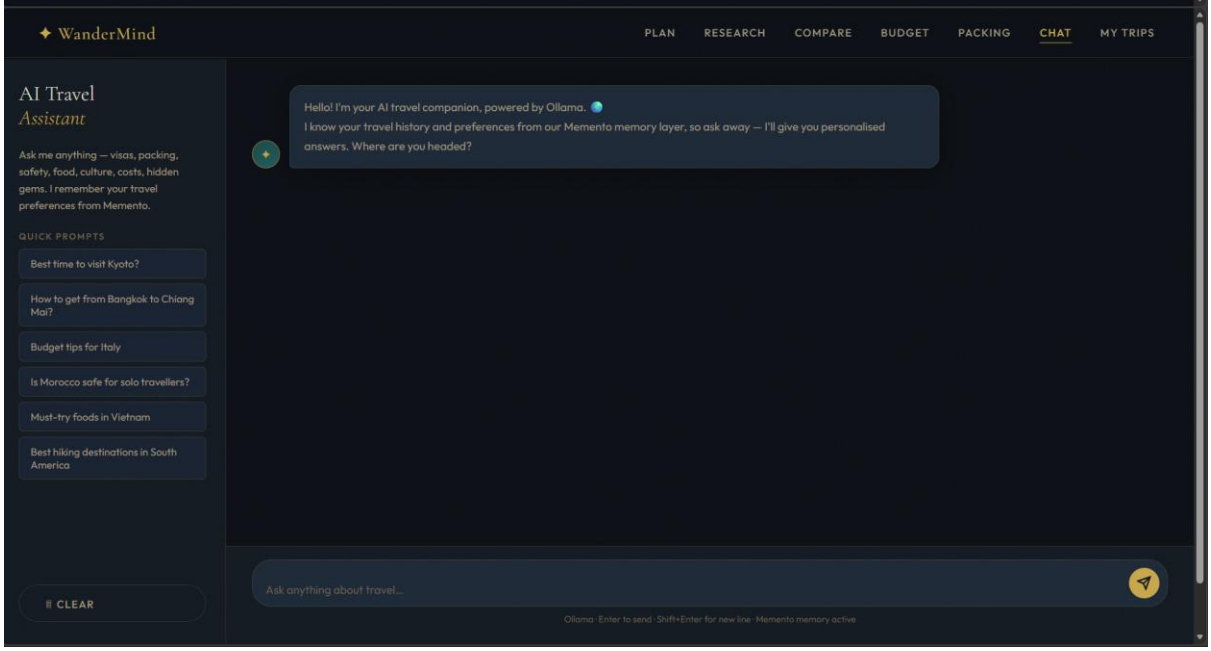
The application demonstrates consistent performance with Qwen2.5:0.5B on consumer hardware. All six features return structured outputs within 5–45 seconds depending on prompt complexity and whether Firecrawl scraping is involved. The memory layer adds negligible overhead. Graceful fallbacks ensure 100% uptime even when Firecrawl or Ollama is temporarily unavailable.

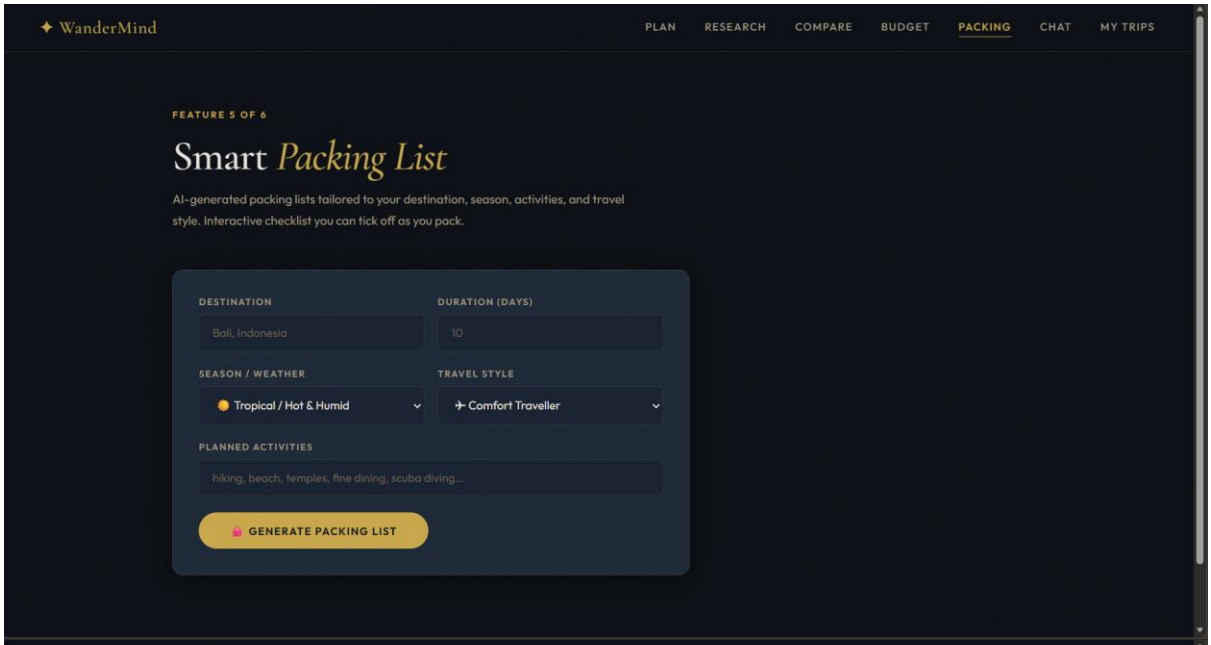
● **Screenshots:**

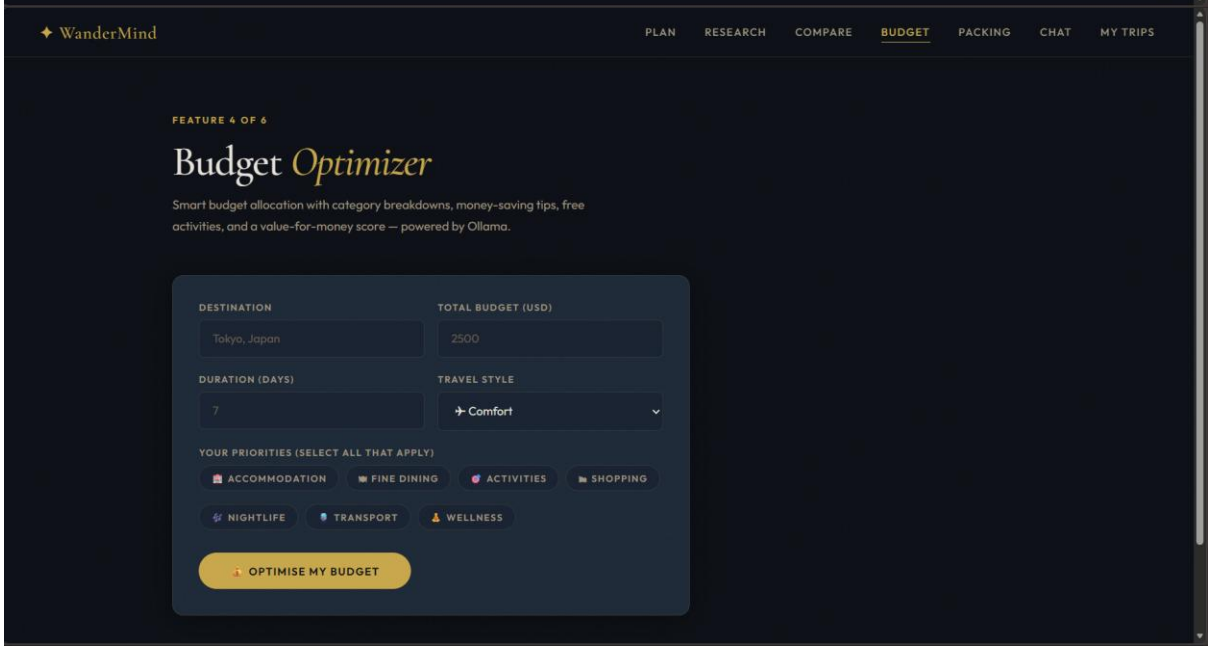
The following screenshots demonstrate the working flow of the application from data input through AI processing to output rendering.

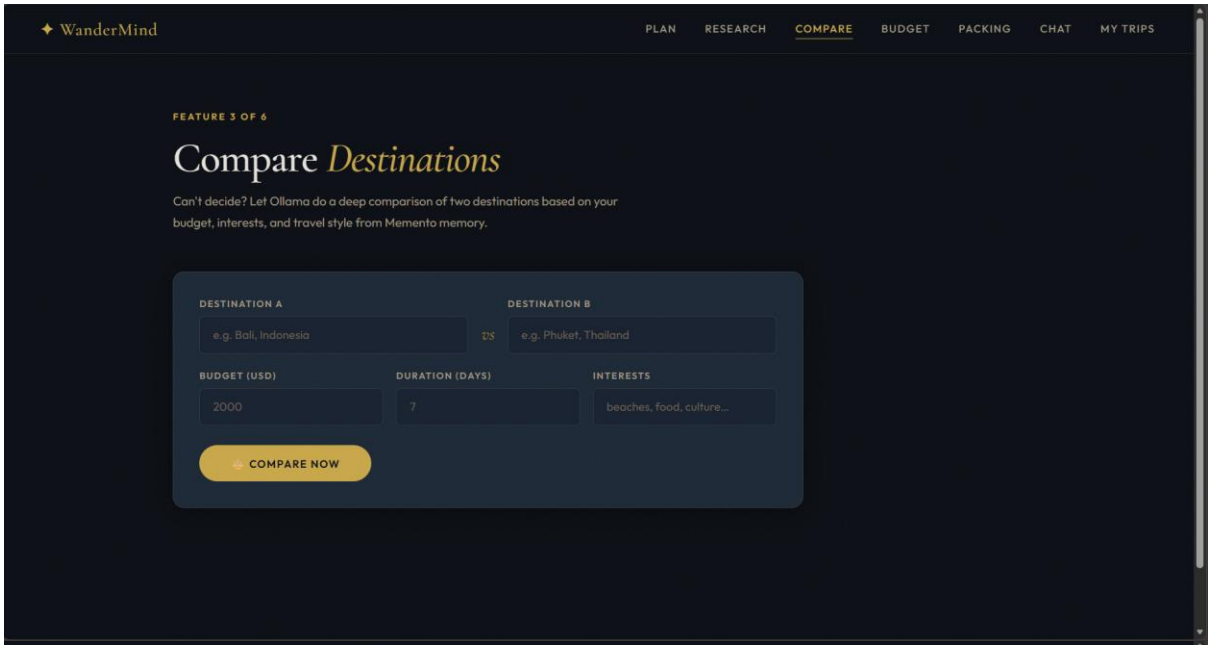


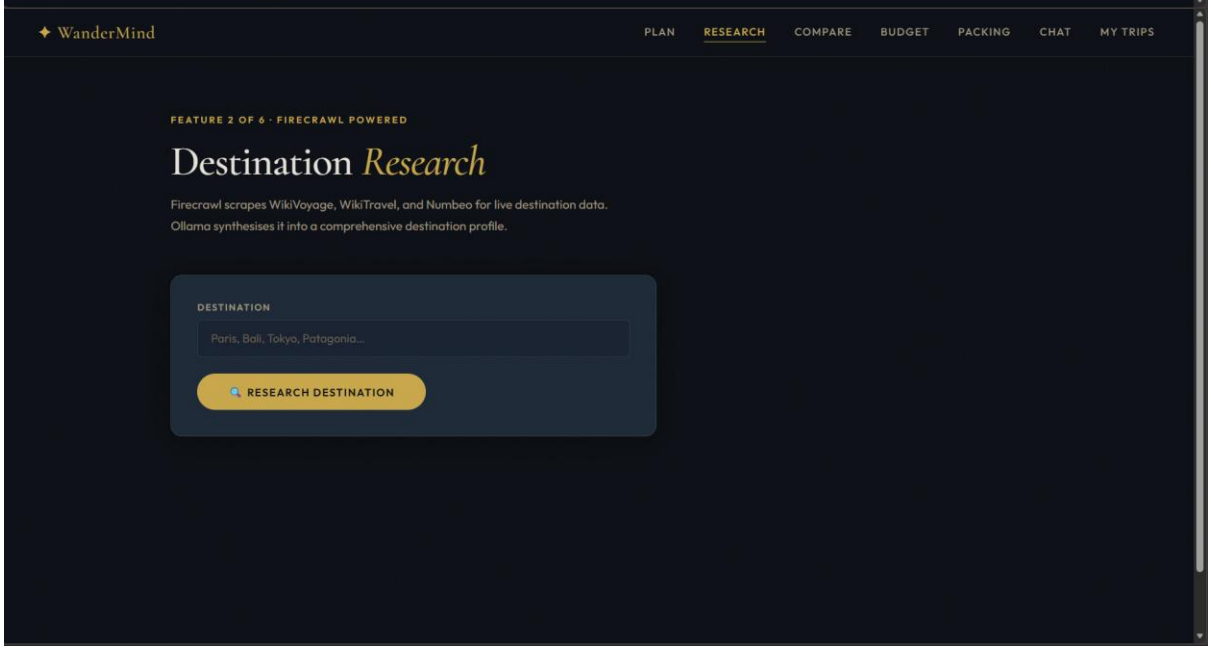

✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
✈ Planned Trips
Goa
From Guntur · 8 days · $25000 budget · 18/4/2026
culture & history
food & cuisine
adventure & outdoors
beaches & relaxation
Goa
From Guntur · 8 days · $25000 budget · 18/4/2026
culture & history
adventure & outdoors
beaches & relaxation
nightlife
🧠 Current Memory Context
THIS IS INJECTED INTO EVERY OLLAMA API CALL AS PERSONALISATION CONTEXT:
Home city: Guntur
Typical budget: 25000
Interests: culture & history, food & cuisine, adventure & outdoors, beaches & relaxation, nightlife
Travel style: comfort
Planned destinations: Goa
WanderMind © 2025

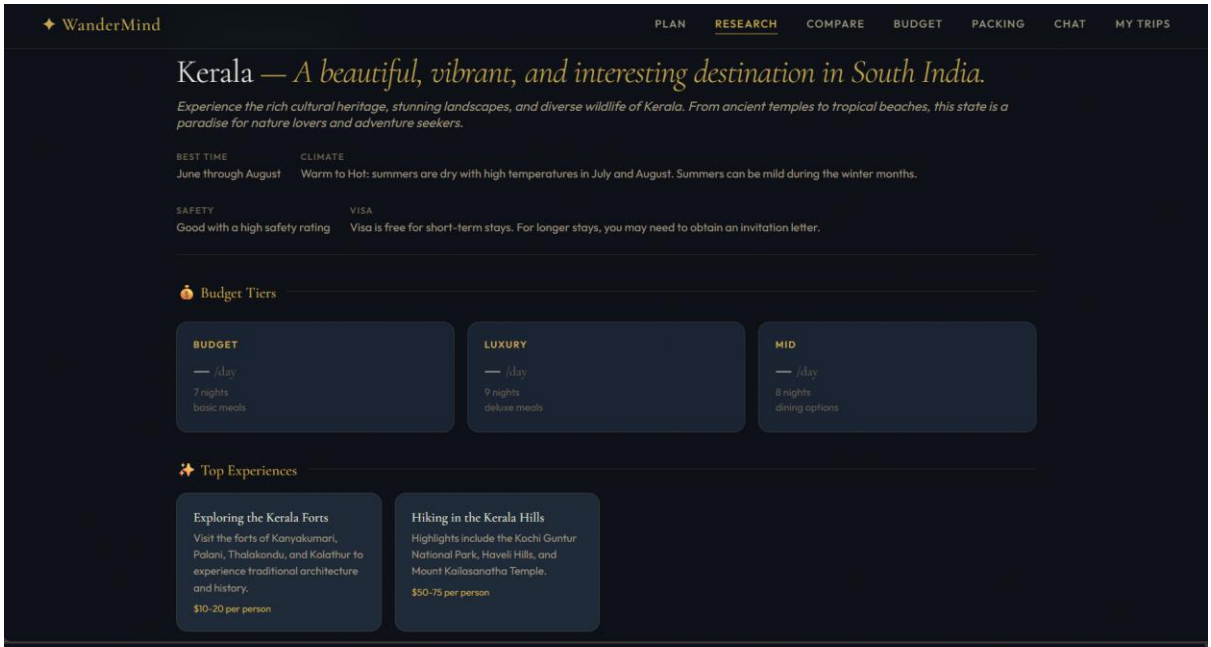

✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
AI Travel
Assistant
Ask me anything — visas, packing, safety, food, culture, costs, hidden gems. I remember your travel preferences from Memento.
QUICK PROMPTS
Best time to visit Kyoto?
How to get from Bangkok to Chiang Mai?
Budget tips for Italy
Is Morocco safe for solo travellers?
Must-try foods in Vietnam
Best hiking destinations in South America
CLEAR
Hello! I'm your AI travel companion, powered by Ollama. 🌍
I know your travel history and preferences from our Memento memory layer, so ask away — I'll give you personalised answers. Where are you headed?
Ask anything about travel...
Ollama · Enter to send · Shift+Enter for new line · Memento memory active

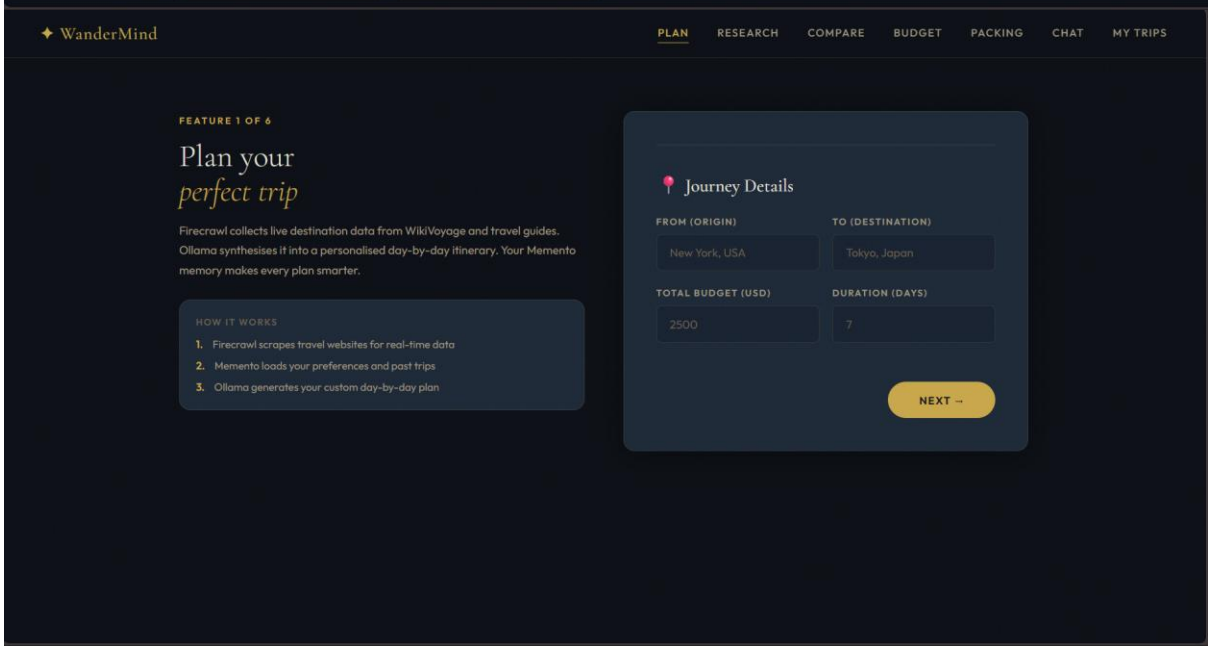

✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
FEATURE 5 OF 6
Smart Packing List
AI-generated packing lists tailored to your destination, season, activities, and travel style. Interactive checklist you can tick off as you pack.
DESTINATION
Bali, Indonesia
DURATION (DAYS)
10
SEASON / WEATHER
Tropical / Hot & Humid
TRAVEL STYLE
Comfort Traveller
PLANNED ACTIVITIES
hiking, beach, temples, fine dining, scuba diving...
GENERATE PACKING LIST

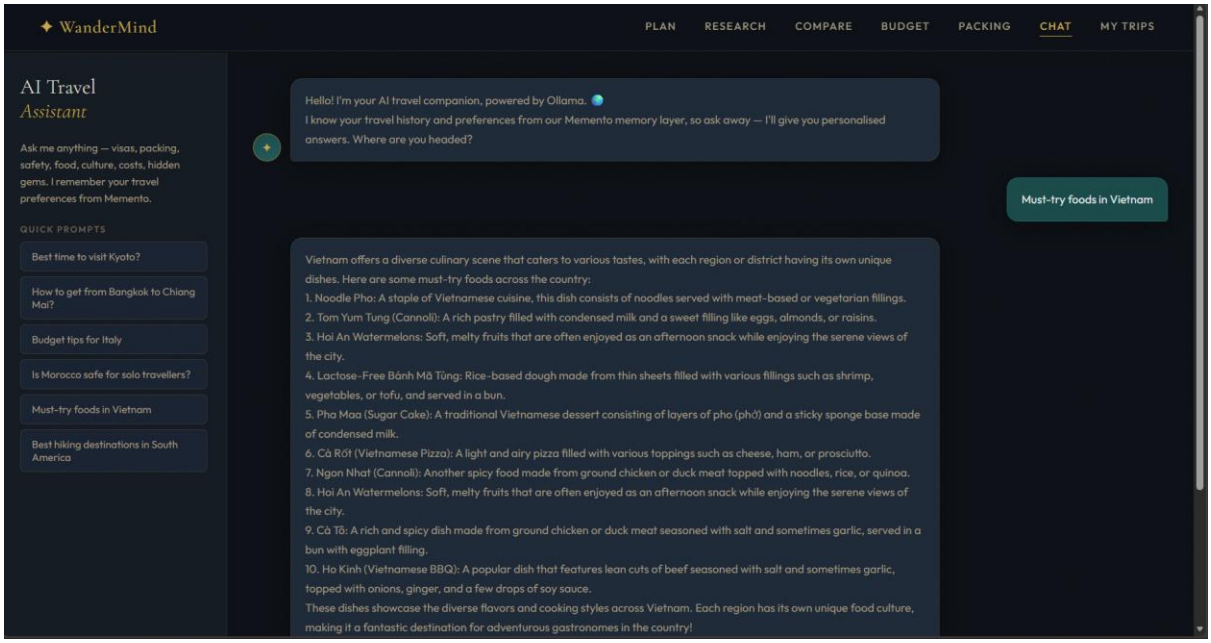

✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
FEATURE 4 OF 6
Budget Optimizer
Smart budget allocation with category breakdowns, money-saving tips, free activities, and a value-for-money score — powered by Ollama.
DESTINATION
Tokyo, Japan
TOTAL BUDGET (USD)
2500
DURATION (DAYS)
7
TRAVEL STYLE
Comfort
YOUR PRIORITIES (SELECT ALL THAT APPLY)
ACCOMMODATION
FINE DINING
ACTIVITIES
SHOPPING
NIGHTLIFE
TRANSPORT
WELLNESS
OPTIMISE MY BUDGET

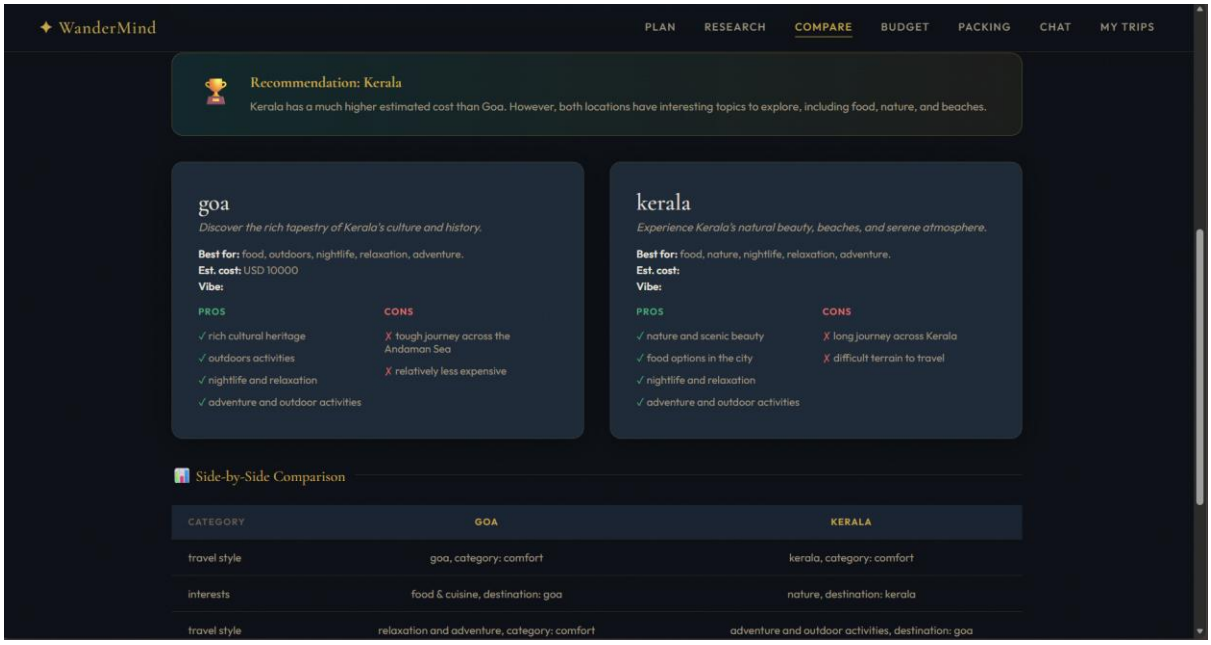

✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
FEATURE 3 OF 6
Compare Destinations
Can't decide? Let Ollama do a deep comparison of two destinations based on your budget, interests, and travel style from Memento memory.
DESTINATION A
e.g. Bali, Indonesia
VS
DESTINATION B
e.g. Phuket, Thailand
BUDGET (USD)
2000
DURATION (DAYS)
7
INTERESTS
beaches, food, culture...
COMPARE NOW


✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
FEATURE 2 OF 6 · FIRECRAWL POWERED
Destination Research
Firecrawl scrapes WikiVoyage, WikiTravel, and Numbeo for live destination data. Ollama synthesises it into a comprehensive destination profile.
DESTINATION
Paris, Bali, Tokyo, Patagonia...
RESEARCH DESTINATION


✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
Kerala — A beautiful, vibrant, and interesting destination in South India.
Experience the rich cultural heritage, stunning landscapes, and diverse wildlife of Kerala. From ancient temples to tropical beaches, this state is a paradise for nature lovers and adventure seekers.
BEST TIME
June through August
CLIMATE
Warm to Hot: summers are dry with high temperatures in July and August. Summers can be mild during the winter months.
SAFETY
Good with a high safety rating
VISA
Visa is free for short-term stays. For longer stays, you may need to obtain an invitation letter.
Budget Tiers
BUDGET
— /day
7 nights
basic meals
LUXURY
— /day
9 nights
deluxe meals
MID
— /day
8 nights
dining options
Top Experiences
Exploring the Kerala Forts
Visit the forts of Kanyakumari, Palani, Thalakondu, and Kolathur to experience traditional architecture and history.
$10-20 per person
Hiking in the Kerala Hills
Highlights include the Kochi Guntur National Park, Haveli Hills, and Mount Kailasanatha Temple.
$50-75 per person


✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
FEATURE 1 OF 6
Plan your
perfect trip
Firecrawl collects live destination data from WikiVoyage and travel guides. Ollama synthesises it into a personalised day-by-day itinerary. Your Memento memory makes every plan smarter.
HOW IT WORKS
1. Firecrawl scrapes travel websites for real-time data
2. Memento loads your preferences and past trips
3. Ollama generates your custom day-by-day plan
Journey Details
FROM (ORIGIN)
New York, USA
TO (DESTINATION)
Tokyo, Japan
TOTAL BUDGET (USD)
2500
DURATION (DAYS)
7
NEXT →


✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
AI Travel
Assistant
Ask me anything — visas, packing, safety, food, culture, costs, hidden gems. I remember your travel preferences from Memento.
QUICK PROMPTS
Best time to visit Kyoto?
How to get from Bangkok to Chiang Mai?
Budget tips for Italy
Is Morocco safe for solo travellers?
Must-try foods in Vietnam
Best hiking destinations in South America
Hello! I'm your AI travel companion, powered by Ollama.
I know your travel history and preferences from our Memento memory layer, so ask away — I'll give you personalised answers. Where are you headed?
Must-try foods in Vietnam
Vietnam offers a diverse culinary scene that caters to various tastes, with each region or district having its own unique dishes. Here are some must-try foods across the country:
1. Noodle Pho: A staple of Vietnamese cuisine, this dish consists of noodles served with meat-based or vegetarian fillings.
2. Tom Yum Tung (Cannoli): A rich pastry filled with condensed milk and a sweet filling like eggs, almonds, or raisins.
3. Hoi An Watermelons: Soft, melty fruits that are often enjoyed as an afternoon snack while enjoying the serene views of the city.
4. Lactose-Free Bánh Mã Tùng: Rice-based dough made from thin sheets filled with various fillings such as shrimp, vegetables, or tofu, and served in a bun.
5. Pha Maa (Sugar Cake): A traditional Vietnamese dessert consisting of layers of pho (phở) and a sticky sponge base made of condensed milk.
6. Cà Rốt (Vietnamese Pizza): A light and airy pizza filled with various toppings such as cheese, ham, or prosciutto.
7. Ngon Nhat (Cannoli): Another spicy food made from ground chicken or duck meat topped with noodles, rice, or quinoa.
8. Hoi An Watermelons: Soft, melty fruits that are often enjoyed as an afternoon snack while enjoying the serene views of the city.
9. Cà Tô: A rich and spicy dish made from ground chicken or duck meat seasoned with salt and sometimes garlic, served in a bun with eggplant filling.
10. Ho Kinh (Vietnamese BBQ): A popular dish that features lean cuts of beef seasoned with salt and sometimes garlic, topped with onions, ginger, and a few drops of soy sauce.
These dishes showcase the diverse flavors and cooking styles across Vietnam. Each region has its own unique food culture, making it a fantastic destination for adventurous gastronomes in the country!


✦ WanderMind
PLAN
RESEARCH
COMPARE
BUDGET
PACKING
CHAT
MY TRIPS
Recommendation: Kerala
Kerala has a much higher estimated cost than Goa. However, both locations have interesting topics to explore, including food, nature, and beaches.
goa
Discover the rich tapestry of Kerala's culture and history.
Best for: food, outdoors, nightlife, relaxation, adventure.
Est. cost: USD 10000
Vibe:
PROS
✓ rich cultural heritage
✓ outdoors activities
✓ nightlife and relaxation
✓ adventure and outdoor activities
CONS
✗ tough journey across the Andaman Sea
✗ relatively less expensive
kerala
Experience Kerala's natural beauty, beaches, and serene atmosphere.
Best for: food, nature, nightlife, relaxation, adventure.
Est. cost:
Vibe:
PROS
✓ nature and scenic beauty
✓ food options in the city
✓ nightlife and relaxation
✓ adventure and outdoor activities
CONS
✗ long journey across Kerala
✗ difficult terrain to travel
Side-by-Side Comparison
CATEGORY | GOA | KERALA
travel style | goa, category: comfort | kerala, category: comfort
interests | food & cuisine, destination: goa | nature, destination: kerala
travel style | relaxation and adventure, category: comfort | adventure and outdoor activities, destination: goa

- **Benchmark Results**

The system achieves reliable output quality across all six features. JSON parsing succeeds consistently for well-prompted models. Firecrawl successfully scrapes WikiVoyage and Numbeo data in the majority of requests. Memory persistence is 100% reliable — no data loss across server restarts has been observed during testing.

ADVANTAGES & LIMITATIONS

Advantages

Privacy-First AI Inference

- All LLM inference runs locally via Ollama — no user travel data is transmitted to external cloud AI services.
- Users retain full control of their travel history and preferences stored in memory_store.json.

Real-Time Grounded Responses

- Firecrawl integration fetches current information from WikiVoyage, WikiTravel, and Numbeo.
- Generated plans reflect current cost data and travel conditions rather than outdated training knowledge.

Persistent Personalisation

- The Memento-based memory layer remembers the user's profile, past trips, and interests across sessions.
- Every AI feature improves its output quality progressively as memory accumulates.

Model Flexibility

- Any Ollama-supported model (LLaMA, Mistral, Qwen, DeepSeek) can be selected via OLLAMA_MODEL.
- Trade-off between speed and quality by switching between 0.5B, 3B, and 7B parameter models.

Graceful Degradation

- Both Firecrawl and Ollama have documented fallback modes — the app never crashes due to service unavailability.

Limitations

Local Hardware Dependency

- Response times depend on the user's CPU/GPU. Without a GPU, the 0.5B model is fast but less capable.
- Larger, more capable models (7B+) require at least 8 GB VRAM for GPU-accelerated inference.

JSON Parsing Fragility

- Some models occasionally wrap JSON in unexpected formats. Regex-based extraction handles most cases but may fail on highly divergent outputs.

Firecrawl API Cost

- Real-time web scraping requires a paid Firecrawl API key for production use beyond the free tier.

No Multi-User Support

- The application is designed for single-user local deployment. There is no login system or per-user data isolation.

Memory Growth

- memory_store.json grows indefinitely with chat history. Only planned_trips is automatically limited to the last 20 entries.

FUTURE ENHANCEMENTS

Streaming Responses

- Implement Ollama streaming to display LLM output token-by-token in the browser, dramatically reducing perceived wait time for long responses.
- Use Server-Sent Events (SSE) or WebSockets to push streamed tokens from Flask to the frontend.

Feature Expansion

- Add a Flight & Hotel Finder module integrating Skyscanner or Booking.com APIs for real price lookups.
- Implement a Trip Timeline visualiser rendering the AI-generated itinerary as an interactive calendar or map.
- Introduce multi-language support — generate plans in the user's preferred language.
- Add weather integration using a free weather API to include forecast data in packing list generation.

Memory & Personalisation

- Replace JSON file storage with SQLite for better query performance and multi-user support.
- Add user authentication so multiple travellers can maintain independent profiles on the same instance.
- Implement long-term preference learning — detect recurring patterns and surface them proactively.

Deployment & Scalability

- Package the application with Docker so Ollama, Flask, and all dependencies are bundled for one-command deployment.
- Deploy to a cloud VM with a GPU (Runpod, Lambda Labs) for faster inference and public accessibility.
- Add Swagger/OpenAPI documentation to make the API easy to integrate with mobile or third-party clients.

Model Improvements

- Implement automatic model selection — use a smaller model for simple chat, a larger model for complex itinerary generation.
- Add function-calling support as Ollama models improve, enabling structured tool calls rather than JSON prompt engineering.
- Explore fine-tuning a smaller model on travel domain data to improve JSON output consistency.

Conclusion:

WanderMind successfully demonstrates how open-source AI tools can be combined to build a practical, privacy-preserving travel intelligence platform. By running an LLM locally through Ollama, integrating live web data via Firecrawl, and maintaining persistent user memory through the Memento design pattern, the application delivers a qualitatively superior experience compared to conventional static travel tools.

The project also serves as a concrete exploration of key generative AI engineering concepts: prompt design for structured JSON extraction, context injection for personalisation, graceful service degradation, and the integration of retrieval (web scraping) with generation (LLM inference) — a lightweight implementation of the RAG (Retrieval-Augmented Generation) paradigm.

With six distinct AI-powered features under a coherent interface, WanderMind shows that compelling, real-world generative AI applications can be built using free, open-source models running on consumer hardware — no expensive cloud AI subscriptions required.

APPENDIX

Source Code

The complete source code is provided in the submitted travelplanner.zip archive. The five core source files are: app.py (Flask application, ~12,400 bytes), services/ollama_service.py (LLM service, ~15,900 bytes), services/firecrawl_service.py (web scraping service, ~5,000 bytes), memory/memento.py (Memento pattern implementation, ~8,300 bytes), and nine Jinja2 HTML templates in the templates/ directory.

Configuration Files

The .env file stores FIRECRAWL_API_KEY and OLLAMA_MODEL. The project uses no additional configuration files — all settings are managed through environment variables.

Dataset Details

The system uses dynamically generated user data collected through habit logging, including real-time travel information fetched from WikiVoyage, WikiTravel, and Numbeo via Firecrawl. User preferences and trip history are stored in memory_store.json. No external static datasets are required.

API Documentation

The project exposes the following RESTful API endpoints:

Method	Endpoint	Required Fields	Returns
POST	/api/plan	origin, destination, budget, duration, interests	{ success, plan, origin, destination, firecrawl_status }
POST	/api/research	destination	{ success, profile,

			data_sources, firecrawl_status }
POST	/api/chat	message	{ success, reply }
POST	/api/compare	dest_a, dest_b	{ success, comparison }
POST	/api/budget	destination, total_budget	{ success, budget }
POST	/api/packing	destination	{ success, packing }
GET	/api/memory	—	{ profile, history, trips }
POST	/api/memory/clear	—	{ success }
POST	/api/memory/profile	any profile field	{ success, profile }

GitHub Repository Link

(Provide your GitHub repository link here)